Tarlac State University
College of Computer Studies
Bachelor of Science in Computer Science
Web Programming

Simulation of CPU Scheduling Algorithms

A Project

Submitted to the Faculty of the College of Computer Studies

San Isidro Campus, Tarlac City,

Tarlac State University

In Partial Fulfillment of the Requirements for the Operating System

Academic Year 2025-2026

Submitted By:
Agudo, Nathalie Joy Q.

Jo Anne Cura
Subject Instructor

Date of submission: May, 2026

Table of Contents

I.Introduction

In modern operating systems, achieving optimal Central Processing Unit (CPU) utilization is a must. The CPU functions as the computer's brain as it processes instructions at extremely fast speeds. However, it can execute only one process at a time while multiple processes require access to it. As such, the operating system needs a method to decide which process gets to use the CPU and when. The process of making these decision functions is called CPU scheduling (GeeksforGeeks, 2026).

CPU scheduling decides how the processes would be executed in order. The system functions by using a ready queue which serves as a directory for all processes that have been stored in memory and are currently waiting to receive CPU processing time. The CPU scheduler examines the queue to select the next process for execution according to predefined rules which act as an algorithm. The various algorithms utilize distinct rule sets. Some algorithms execute processes based on their arrival sequence while others focus on shorter tasks and some utilize fixed time windows for process switching and others allocate processing power to critical tasks. Each approach has its own strengths and weaknesses depending on the situation.

Scheduling algorithms divide into two distinct categories (Seemakuthi et al., 2015). The non-preemptive algorithms allow processes to execute until they complete their tasks. Preemptive algorithms enable interruption of running processes which allows another process to take control of the CPU.

Now, the main purpose of this case study is to design and develop a program that puts those algorithms into code so that the behavior of each one can be observed directly. By entering real process data and seeing the output, it becomes much easier to understand how each algorithm works and how they differ from one another in terms of performance.

The program that will be developed for this case study simulates six scheduling algorithms: First-Come, First-Served, Shortest Job First, Shortest Remaining Time, Round Robin, Priority Scheduling, and Priority Scheduling with Round Robin. The program accepts user input for each process, including arrival time, burst time, priority, and time quantum where needed. After simulating the selected algorithm, it displays a Gantt chart showing the order and timing of process execution, along with a table showing the waiting time and turnaround time for each process, and their computed averages.

The objectives of this program are straightforward. First, to simulate all six scheduling algorithms. Second, to show the results clearly through a Gantt chart and a summary table that includes waiting time, turnaround time, and their averages. By designing and developing a program that executes the six scheduling algorithms, this case study provides a realization of the theoretical scheduling algorithms.

II.Body

When multiple processes exist, the CPU can only execute one process at a time. To ensure an organized execution, the CPU scheduler determines which process runs, the order, and how long they run. The following program presents the differing scheduling algorithms and their distinct approach in handling processes. Said scheduling algorithms are as follows: FCFS, SJF (Non-Preemptive), SRT, Round Robin, Priority (Non-Preemptive & Preemptive), and Priority + Round Robin. In determining the performance of the CPU, waiting time, turnaround time, and completion time is computed.

A. Main Execution Loop

```
# main program entry point
def main():
    print("\nChoose Scheduling Algorithm:")
    print("1. FCFS")
    print("2. SJF (Non-Preemptive)")
    print("3. SRT (Preemptive SJF)")
    print("4. Round Robin")
    print("5. Priority Scheduling (Non-Preemptive)")
```

```
    print("6. Priority Scheduling (Preemptive)")
    print("7. Priority + Round Robin")

    while True:
        try:
            choice = int(input("Enter choice: "))
            if choice in range(1, 8):
                break
            print("Please enter a number between 1 and 7.")
        except ValueError:
            print("Please enter a valid integer.")

    processes = get_input(choice)
    scheduler = Scheduler(processes)

    if choice == 1:
        scheduler.fcfs()
    elif choice == 2:
        scheduler.sjf_non_preemptive()
    elif choice == 3:
        scheduler.srt()
    elif choice == 4:
        while True:
            try:
                quantum = int(input("\nEnter Time Quantum: "))
                break
            except ValueError:
                print("Please enter a valid integer.")
        scheduler.round_robin(quantum)
    elif choice == 5:
        scheduler.priority_non_preemptive()
    elif choice == 6:
        scheduler.priority_preemptive()
    elif choice == 7:
        while True:
            try:
                quantum = int(input("Enter Time Quantum: "))
                break
            except ValueError:
                print("Please enter a valid integer.")
        scheduler.priority_round_robin(quantum)
    else:
        print("Invalid choice")


# run simulation loop
while True:
    main()
    print("\nRun another simulation?")
    print("1. Yes")
    print("2. No")

    while True:
        try:
            again = int(input("Enter choice: "))
            if again in [1, 2]:
                break
            print("Please enter 1 or 2 only.")
        except ValueError:
            print("Please enter a valid integer.")

    if again == 2:
        print("Exiting program...")
        break
```

Figure 1. Main Execution Loop of CPU Scheduling Algorithm Program

The <main()> function serves as the entry point of the entire program. It presents the menu, collects the user's scheduling algorithm choice, and simulates the appropriate scheduling algorithm.

Firstly, the function prints the menu of seven available scheduling algorithms numbered. A <while> loop wrapped in a <try-except> block prompts the user to enter their choice to constrain inputted integers only from 1 to 7.

Once valid a <choice> is passed into <get_input()> so that it knows whether to prompt for priority values, and the returned list of <Process> objects is stored in <processes>. Thereafter, a <Scheduler> object is instantiated using <processes>, which is what all scheduling algorithm functions call.

Then, a series of <if-elif> conditions so that only choices 4 and 7 has an additional <while> loop which prompts the user for a time quantum before calling the function. This is done so since both Round Robin and Priority + Round Robin require it to simulate the quantum time slice. There is also a <try-except> block to handle invalid entries for time quantum.

Finally, <main()> is called at the bottom of the file to run the program. It prompts the user if they want to run another scheduling algorithm or exit the program.

B. User Input

```
# get user input for processes
def get_input(choice):
    needs_priority = choice in [5, 6, 7]

    while True:
        try:
            n = int(input("Enter number of processes (>=3): "))
            if n >= 3:
                break
            print("Minimum is 3 processes.")
        except ValueError:
            print("Please enter a valid integer.")

    processes = []

    for i in range(n):
        print(f"\nProcess {i + 1}")
        pid = input("Process Identifier: ")

        while True:
            try:
                bt = int(input("Burst Time: "))
                at = int(input("Arrival Time: "))

                if needs_priority:
                    pr = int(input("Priority (Lower Value = Higher
Priority): "))
                else:
                    pr = 0

                break
            except ValueError:
                print("Please enter valid integers.")

        processes.append(Process(pid, at, bt, pr))

    return processes
```

Figure 2. Input Validation and Process Data Collection

The <get_input()> function collects all process data entered from the user, returning a list of <Process> objects that the <Scheduler> will use for simulation. It accepts parameter <choice> as the user's chosen scheduling algorithm from the main menu.

The <needs_priority> is a boolean that checks whether the <choice> of the user belongs to priority-based scheduling algorithms so that it only asks the user for a priority value when the chosen algorithm actually requires it.

A <while> loop prompts the user to enter the number of processes as <n>. The input is checked in the <try-except> block to prevent non-integer inputs and also

constraint the number of processes not less than 3. Once valid the loop breaks and then prompts the user for each process's attributes.

The <processes[]> list is initialized as empty, and a <for> loop runs <n> times where each iteration the process ID is asked. Inside the loop, another <while> loop within a <try-except> block to prompt the user to enter the burst time <bt> and arrival time <at>.

If <needs_priority> is True, the user is further prompted for a priority value <pr> otherwise the priority is set to 0. Once all attributes are collected without error, the inner loop breaks and a <Process> object is instantiated and then appended to <processes>. Once all tge processes have been collected, <get_input()> returns the populated <processes> list to <main()> for scheduling.

C. Class Process and Class Scheduler

```
import matplotlib.pyplot as plt

class Process:
    # holds individual process data
    def __init__(self, pid, arrival, burst, priority=0):
        self.pid = pid
        self.arrival = arrival
        self.burst = burst
        self.priority = priority
        self.waiting = 0
        self.turnaround = 0
        self.completion = 0


class Scheduler:
    # manages all scheduling algorithms
    def __init__(self, processes):
        self.processes = processes
        self.gantt = []

    # reset all process metrics
    def reset(self):
        self.gantt = []
        for p in self.processes:
            p.waiting = 0
            p.turnaround = 0
            p.completion = 0

    # compute waiting and turnaround
    def calculate_metrics(self, p, completion_time):
        p.completion = completion_time
        p.turnaround = p.completion - p.arrival
        p.waiting = p.turnaround - p.burst
```

Figure 3. Class Process and Class Scheduler

In class The <matplotlib.pyplot> is imported to be later used by <plot_gantt()>. Through this, the program will later visually represent how processes are executed based on the scheduling algorithm chosen by the user.

The <class Process> defines the blueprint of every process in the program. Each Process object created is given a process ID <pid> as a label which will be used for tracking execution in Gantt chart and the resulting table. It also includes <arrival> time to determine when the process enters the program, this is especially important for scheduling algorithms like SRT and RR where the ready queue changes over time based on when they arrive.

Another is the <burst> time which represents the total CPU time the process finishes, or how long the process occupies the CPU. Then the <priority> which determines the urgency of a process, and in this program, the lower the value the higher the priority. The priority defaults to 0 because algorithms like FCFS, SJF, SRT and RR ignore priority, unlike priority-based algorithms.

The class process also has <waiting> time, which is the time a process spends waiting in the ready queue, as well as the <turnaround> time which measures the total time from arrival to completion. These two, waiting and turnaround time, are not input by results that will be used to evaluate how efficient the chosen scheduling algorithm is. Ultimately, to calculate the two, there is a need for <completion> time which is the exact time when a process finishes executing.

The <class Scheduler> is the core of the program. While <class Process> defines the attributes that a process carries, the <class Scheduler> defines how the processes are handled based on the chosen scheduling algorithm. Firstly, <class Process> receives a list of Process objects that the user inputted and stores it within <self.processes>. Said list becomes what every scheduling algorithm uses as a reference for deciding the execution order, computing waiting and turnaround time, as well as creating the Gantt chart and table. Now, there are two helper functions for <class Scheduler> first is <reset()> and second is <calculate_metrics()>

First, as the <self.processes> and <self.gant> are shared across the entire Scheduler object, if a scheduling algorithm runs and another algorithm runs thereafter, the latter's data would overlap with the former's Gantt chart and computation. Subsequently, the result for waiting time and turnaround time would also conflict. Hence, the <reset()> is needed to empty the already populated Gantt chart and loops through the <self.processes> to set each processes' waiting and turnaround time back to 0. This ensures that the results and Gantt chart produced solely belong to the current chosen scheduling algorithm, and not conflicting with previous runs.

Second, since each scheduling algorithms required to calculate the metric values, such as the completion time, turnaround time, and waiting time, the <calculate_metrics()> is used to compute said metrics by accepting two parameters, (a) <p> as the process itself; (b) <completion_time> which is the exact time a process finishes execution, and thereafter used as the <p.completion>. On one hand, Turnaround Time is calculated as Completion Time - Arrival Time, on the other hand, Waiting Time is calculated as Turnaround Time - Burst Time (Silberschatz et al., 2013). These computations are reused throughout each of the scheduling algorithms for evaluation of the CPU performance.

C.1. First-Come, First-Served (FCFS)

```
    # first come first served
    def fcfs(self):
        # sort by arrival
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0

        for p in processes:
            # handle idle time
            if time < p.arrival:
                self.gantt.append(("IDLE", time, p.arrival))
                time = p.arrival

            # execute process
            start = time
            time += p.burst
            finish = time

            # store results
            self.calculate_metrics(p, finish)
            self.gantt.append((p.pid, start, finish))

        self.display("FCFS")
        self.plot_gantt("FCFS")
```

Figure 4. FCFS Function

The First-Come, First-Served (FCFS) method runs processes based on their arrival time because it executes the first process that enters the ready queue

(Seemakuthi et al., 2015). This method sorts by arrival time, and runs all the process's burst time from start to finish before moving onto the next one. The FCFS scheduling method operates non-preemptively because the executing process keeps a hold of the CPU until it finishes and no process can break this. However, it causes short processes to experience extended waiting times when a long process needs to finish before them.

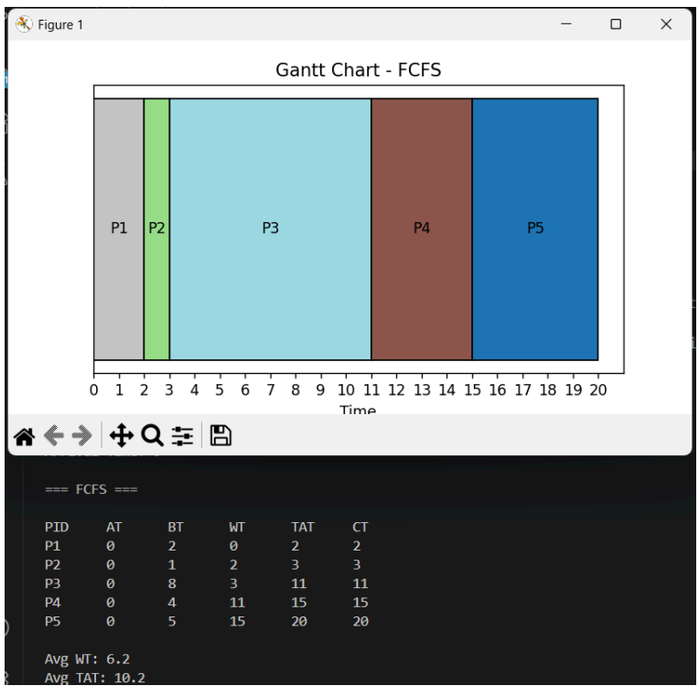
Moving on to the program, for each scheduling algorithm function, the program first runs the <reset()> function to start anew and avoid interference from previous runs. Thereafter, the program simulates the chosen algorithm. For <fcfs()> it's core function relies on using <sorted()>.

The <sorted()> built-in function acquires all the user inputted processes <self.processes> , and then arranges them in ascending order based on their arrival time using <key=get_arrival>. In the case that all processes from P1 to Pn have 0 arrival time, the execution order is as it is. This guarantees that the CPU handles processes in the exact order they enter the system, as is the rule for FCFS. Also, the <time> variable, initialized as 0, represents the CPU clock that advances as the processes are executed.

Then the process is executed, recording its <start> time or when the process execution begins, as well as its <finish> time. Meanwhile, the CPU <time> advances based on the process's burst time.

Finally, the process' turnaround, waiting, and completion time is computed using the <calculate_metrics()>. The Gantt chart function is also populated depending on the arranged processes.

However, if the current time is less than a process's arrival time, there would inevitably be a gap. Hence, for this case the gap on the CPU time is marked as idle in the Gantt Chart, and the time is forwarded to the arrival time of that process.



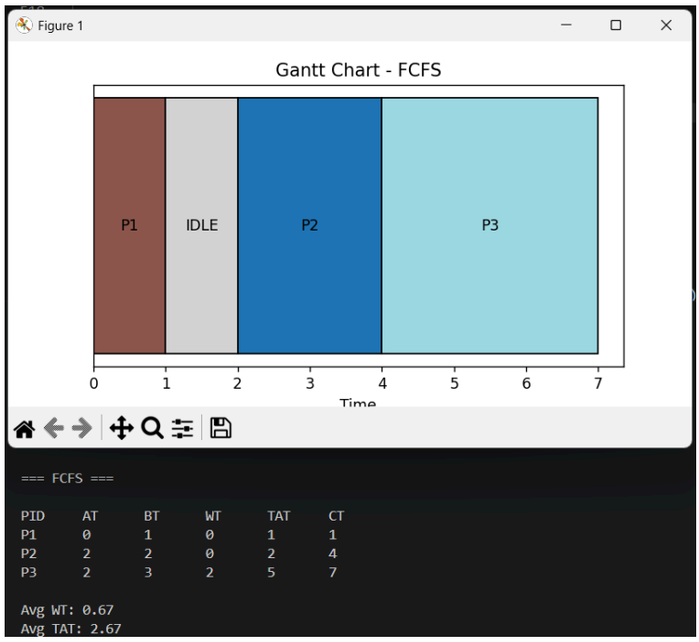


Figure 5. FCFS Test Cases

Test Case 1 and Test Case 2 visualizes the execution of the processes in terms of FCFS wherein the CPU scheduler sorts the processes according to their arrival time, or rather the rule First Come, First Serve. In the case wherein all processes arrived in 0 CPU time, the sorting depended on which process was first inputted by the user, in this case from P1 to P5. It could change depending on the identifier tagged by the user. Additionally, in Case 2 since certain processes arrive late, there is an additional idle time depicted on the Gantt chart.

C.2. Shortest Job First (SJF)

```
  # shortest job first nonpreemptive
    def sjf_non_preemptive(self):
        # initialize tracking
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0
        completed = 0
        n = len(processes)
        ready_queue = []
        visited = [False] * n

        while completed < n:
            # add arrived processes
            for i in range(n):
                if processes[i].arrival <= time and not visited[i]:
                    ready_queue.append(processes[i])
                    visited[i] = True

            # handle idle cpu
            if not ready_queue:
                next_arrival = None
                for i in range(n):
                    if not visited[i]:
                        if next_arrival is None or processes[i].arrival
< next_arrival:
                            next_arrival = processes[i].arrival
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                continue

            # pick shortest job
            shortest = min(ready_queue, key=lambda p: p.burst)
            ready_queue.remove(shortest)

            # run to completion
            start = time
```

```
            time += shortest.burst

            # record metrics
            self.calculate_metrics(shortest, time)
            self.gantt.append((shortest.pid, start, time))

            completed += 1

        self.display("SJF (Non-Preemptive)")
        self.plot_gantt("SJF")
```

Figure 6. SJF Function

The Shortest Job First (SJF) executes the process according to the smallest burst time among all processes in the ready queue (Seemakuthi et al., 2015). SJF operates as a non-preemptive system which requires process execution to continue until its completion, similar to FCFS. SJF does not execute in arrival order because it selects the shortest job available which results in reduced average waiting time. The system experiences delays because shorter tasks continue to arrive, which prevents longer tasks from finishing their work. The problem exists because shorter tasks keep stopping longer tasks from finishing their work.

Now onto the program, after resetting, the processes are sorted by arrival time using <sorted()> and a lambda function. The <time> variable represents the CPU clock and continuously advances as processes execute, the <completed> tracks the finished processes, and the <n> stores the total number of processes. Meanwhile, <ready_queue> lists all processes that have already arrived and are waiting to be executed, and <visited> is a boolean list of visited process.

The <while> loop continuously runs until all <n> processes are finished. On each iteration, it scans through all processes using <range(n)> and checks if a process has an arrival time less than or equal to <time> and has not yet been visited. If both conditions are met, the process is appended to <ready_queue> and is set to True. This is done so that each process is only transferred once and does not get duplicated or re-executed.

Now, the <ready_queue> list represents the processes that are ready and waiting for CPU execution. From this, the scheduler selects the process with the shortest burst time through <min(ready_queue, key=lambda p: p.burst)> which translates to finding the process amongst the <ready_queue> based on their burst time. This selected process is then removed from the <ready_queue> to avoid conflicts, and then the process is executed fully by calculating its start and finish time, which subsequently advances the CPU time based on its burst time.

Finally, once a process is selected as the shortest job, the <calculate_metrics()> function computes its performance metrics and populates the Gantt chart. The while loop continues until <completed> equals <n>.

Additionally, there is a condition that handles CPU idle time. If by chance the <ready_queue> is empty yet there are still unvisited processes, it means that no process has arrived yet at the current time. This represents a gap. In this case, the program finds the lowest arrival time among the unvisited processes using <min(p.arrival for i, p in enumerate(processes) if not visited[i])>, which translates to finding the lowest arrival time amongst the remaining unvisited processes. After this, the Gantt chart is populated with "IDLE" starting from the current time to the next arrival time, and thereafter, the current CPU <time> jumps to <next_arrival>.

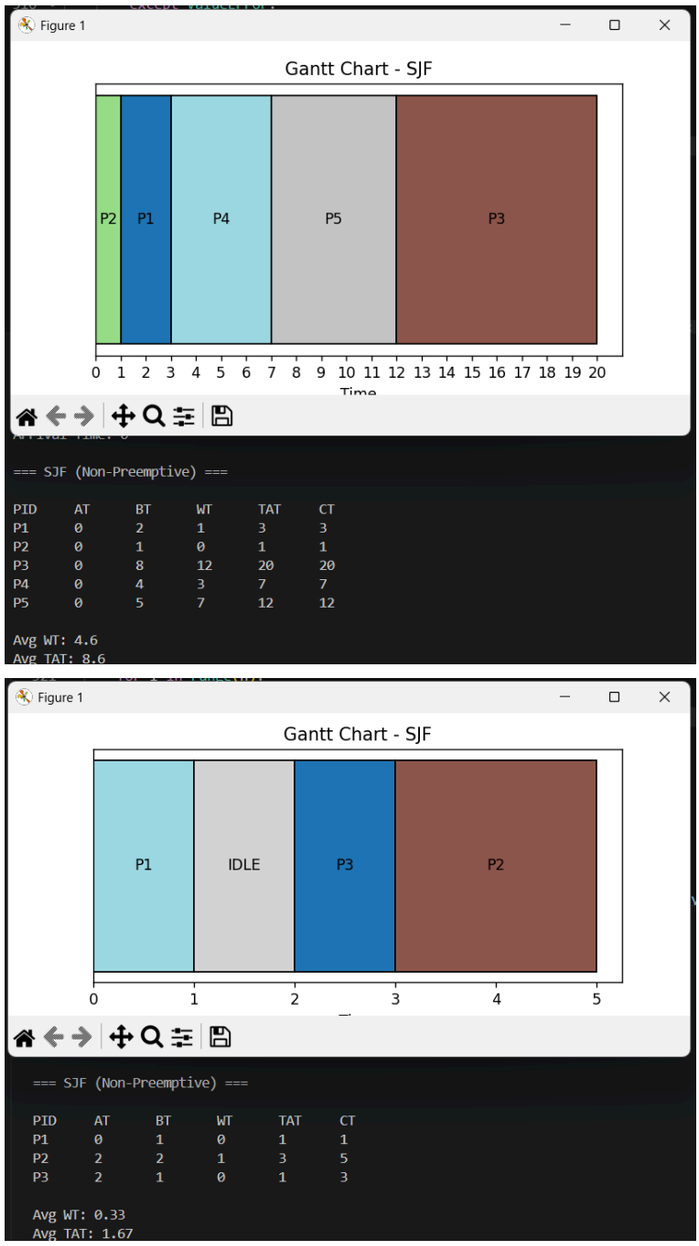


Figure 7. SJF Test Cases

Test Case 1 and Test Case 2 visualizes the execution of the processes in terms of SJF wherein the CPU scheduler sorts the processes according to their arrival time as well as the duration of their burst time. In SJF rule, those with the shortest burst time are immediately attended to. Additionally, in Case 2 since certain processes arrive late, there is an additional idle time depicted on the Gantt chart.

C.3. Shortest Remaining Time (SRT)

```
# shortest remaining time preemptive
def srt(self):
    # setup remaining bursts
    self.reset()
    processes = sorted(self.processes, key=lambda p: p.arrival)
    time = 0
    completed = 0
    n = len(processes)
    remaining_burst = []
    for p in processes:
        remaining_burst.append(p.burst)
    visited = [False] * n
    prev = None

    while completed < n:
        # find ready processes
```

```
            ready_indices = []
            for i in range(n):
                if processes[i].arrival <= time and not visited[i]:
                    ready_indices.append(i)

            # handle idle time
            if not ready_indices:
                next_arrival = None
                for i in range(n):
                    if not visited[i]:
                        if next_arrival is None or processes[i].arrival
< next_arrival:
                            next_arrival = processes[i].arrival
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                continue

            # pick shortest remaining
            best_index = ready_indices[0]
            for i in ready_indices:
                if remaining_burst[i] < remaining_burst[best_index]:
                    best_index = i

            # execute one unit
            p = processes[best_index]

            if prev != best_index:
                self.gantt.append([p.pid, time, time + 1])
            else:
                self.gantt[-1][2] += 1

            remaining_burst[best_index] -= 1
            time += 1
            prev = best_index

            # check for completion
            if remaining_burst[best_index] == 0:
                visited[best_index] = True
                completed += 1
                self.calculate_metrics(p, time)

        self.display("SRT (Preemptive SJF)")
        self.plot_gantt("SRT")
```

Figure 8. SRT Function

The Shortest Remaining Time (SRT), which is the preemptive version of SJF, checks whether a newly arrived process has a shorter remaining time than the currently running one, and if so, it switches immediately (Seemakuthi et al., 2015). The CPU system allows interruptions through process arrivals which have shorter remaining burst times. The scheduling algorithm enables short processes to run immediately without delays caused by longer tasks. However, this faces problems because SJF lets shorter jobs enter the queue which prevents longer tasks from completing their work (Silberschatz et al., 2013).

Moving onto the code, the SRT function first resets Gantt charts and metrics, sorts the processes by arrival time, as well as initializing the CPU time. Now unlike the SJF above which tracks processes through a <ready_queue>, SRT instead uses <remaining_burst> which is a list where each index corresponds to the same index in <processes>, storing their remaining burst time. This is required as SRT does not run a process's burst time all at once, rather it executes tick by tick. Additionally, the <completed> variable acts as a counter of how many processes have been completed, and <n> is the total number of processes. The <visited> boolean list tracks which processes have already finished, while <prev> tracks the index of the process that ran in the previous tick. Later it will be used to detect when a context switch happens.

Next, the <while> loop runs until all <n> processes are completed. Inside the loop, <ready_indices> is first built by scanning through all processes using <range(n)> and collecting the indices of processes that have arrived at or before the current <time> and have not yet been marked as visited. This index-based approach is what allows the

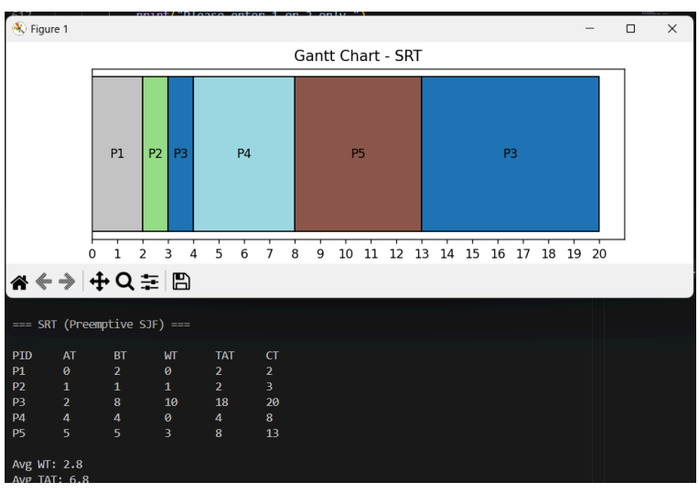
scheduler to look up each process's remaining burst time directly through <remaining_burst>.

If <ready_indices> turns out to be empty, it means no process has arrived yet, representing a CPU idle gap. In this case, the program manually scans through all unvisited processes to find the one with the lowest arrival time, storing it in <next_arrival>. The Gantt chart is then populated with "IDLE" from the current <time> to <next_arrival>, and the CPU <time> jumps forward accordingly.

Once <ready_indices> is confirmed to be non-empty, the scheduler finds the process with the shortest remaining burst time. It starts by assuming the first index in <ready_indices> is the best candidate, stored in <best_index>, then iterates through the rest. If a process at index <i> has a smaller remaining burst than the current <best_index>, it replaces it. The process at <best_index> is then assigned to <p> and selected for execution.

Now, since SRT is preemptive and runs tick by tick, the Gantt chart is handled differently compared to SJF. Instead of appending a new entry every tick, the scheduler checks if <prev> is equal to <best_index>. If they are the same, meaning the same process is still running, the last Gantt entry's end time is extended by one tick through <self.gantt[-1][2] += 1>. If they differ, meaning a context switch has occurred, a new Gantt entry is appended for the newly running process. After this, <remaining_burst[best_index]> is decremented by one, <time> advances by one tick, and <prev> is updated to <best_index>.

Finally, after each tick, the scheduler checks if <remaining_burst[best_index]> has reached zero. If so, the process is marked as finished by setting <visited[best_index]> to <True>, incrementing <completed> by one, and calling <calculate_metrics()> to compute its performance metrics. The <while> loop continues until <completed> equals <n>, meaning all processes have been fully executed.



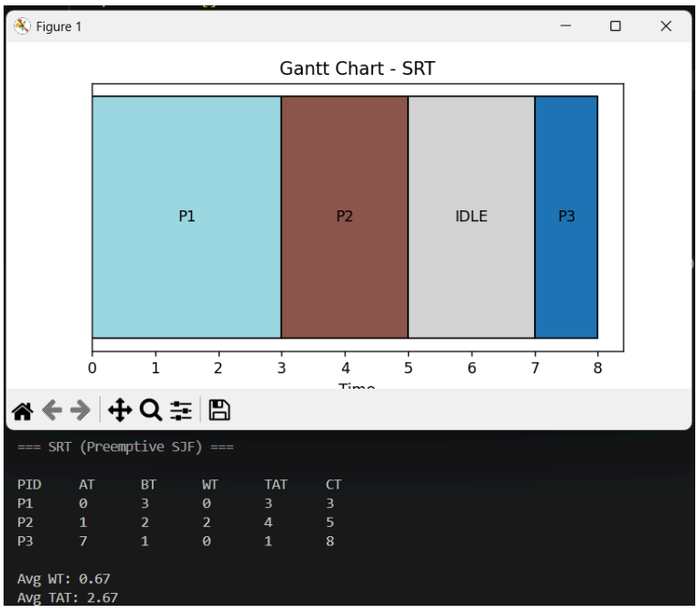


Figure 9. SRT Test Cases

Test Case 1 and Test Case 2 visualizes the execution of the processes in terms of SRT wherein the CPU scheduler sorts the processes according to their arrival time as well as the duration of their burst time. However, unlike the SJF which is non-preemptive, SRT is preemptive. Hence, the CPU scheduler executes the processes tick by tick, not their whole burst time. Which means, in every moment, the CPU scheduler scans those processes in the ready queue, and always chooses the one with the shortest burst time. Additionally, in Case 2 since certain processes arrive late, there is an additional idle time depicted on the Gantt chart.

C.4. Round Robin (RR)

```
    # round robin with quantum
    def round_robin(self, quantum):
        # initialize queues
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0
        completed = 0
        n = len(processes)
        remaining_burst = [p.burst for p in processes]
        visited = [False] * n
        ready_queue = []
        added = [False] * n

        # add initial processes
        for i in range(n):
            if processes[i].arrival <= time:
                ready_queue.append(i)
                added[i] = True

        while completed < n:
            # handle empty queue
            if not ready_queue:
                next_arrival = None
                for i in range(n):
                    if not added[i]:
                        if next_arrival is None or processes[i].arrival
< next_arrival:
                            next_arrival = processes[i].arrival
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                for i in range(n):
                    if processes[i].arrival <= time and not added[i]:
                        ready_queue.append(i)
                        added[i] = True
                continue
```

```
            # get next process
            idx = ready_queue.pop(0)
            p = processes[idx]

            # determine run time
            if remaining_burst[idx] < quantum:
                run_time = remaining_burst[idx]
            else:
                run_time = quantum

            # execute time slice
            start = time
            time += run_time
            remaining_burst[idx] -= run_time

            # record execution
            self.gantt.append((p.pid, start, time))

            # add newly arrived
            for i in range(n):
                if processes[i].arrival <= time and not added[i]:
                    ready_queue.append(i)
                    added[i] = True

            # check completion
            if remaining_burst[idx] == 0:
                visited[idx] = True
                completed += 1
                self.calculate_metrics(p, time)
            else:
                ready_queue.append(idx)

        self.display("Round Robin")
        self.plot_gantt("Round Robin")
```

Figure 10. RR Function

The Round Robin (RR) scheduling algorithm executes processes in repeating order in a fixed time slice called the quantum (Seemakuthi et al., 2015). This algorithm is preemptive since if a process does not finish within its quantum it is interrupted and repositioned back in the ready queue. Through this method, each process is given a fair amount of time or quantum to finish, however, if the quantum assigned is too small, frequent context switching can reduce CPU efficiency.

Onto the program, the RR function initially resets Gantt charts and metrics, sorts the processes by arrival time, as well as initialize the CPU time. It has four main lists to manage the process execution, specifically the <remaining_burst[]> tracks how many burst time each process is left with, <visited[]> is a boolean list tracking which processes have already finished execution <ready_queue[]> is the list of executing in FIFO method, and lastly, <added[]> prevents duplicates of processes that have initially been added in the ready queue. Next, a <for> loop is used to add the initial processes to the <ready_queue[]> that have already arrived depending on the current CPU clock.

Then, a <while> loop is used to run the algorithm of RR, this loop runs until all processes finish. Inside said loop is the main core of the function, which simulates the RR scheduling algorithm. For all processes that are yet included in the <addded[]> list and is less than or equal to the current CPU <time>, they are appended in the <ready_queue[]>.

In the condition that the <ready_queue> is non-empty then the RR scheduling algorithm is simulated as follows, which demonstrates the fairness rule of RR. To do that the <idx> variable marks the first process's index in the <ready_queue[]> using <pop(0)>. After that, using <if> condition, the function checks the chosen process's <remaining_burst[]>. If the burst time is less than the quantum, for example burst time 1 is compared with quantim time slice 2, then <run_time> variable stores the process's remaining burst time, else, it stores the quantum slice time. As whole, it follows the FIFO order and ensures that the processes only runs exact within the quantum limit. Following

that, the CPU clock the process's <start> is recorded and its <remaining_burst[]> time is decremented, plus populating the Gantt Chart.

Since the CPU <time> is updated, before re-running the <while> newly arrived processes are appended in the <ready_queue[]>. Furthermore, the function check is the previous ran process's burst time is equal to 0. If True then it is included in the <visited[]> to prevent duplicate. Also, its performance metrics is calculated using <calculate_metrics(). However, if the process's still has remaining burst time, then it is once again appended to the end of the <ready_queue[]>

For idle handling, a nested <if> condition checks if the <ready_queue[]> and <added[] lists are empty, if so then the CPU clock <time> jumps to the next process arrival time, then immediately logs the idle period in Gantt chart.

Figure 11. RR Test Cases

Test Case 1 and Test Case 2 visualizes the execution of the processes in terms of RR wherein the CPU scheduler sorts the processes according to their arrival time. Unlike SJF and SRT who favour the processes with the shortest burst time, RR abides by the rule of fairness. In other words, each process is only given a pre-defined CPU quantum time, and once again redirected at the back of the ready queue lane. Additionally, in Case 2 since certain processes arrive late, there is an additional idle time depicted on the Gantt chart.

B.5 Priority Scheduling (Non-Preemptive or Preemptive)

Non-Preemptive

```
    # priority nonpreemptive
    def priority_non_preemptive(self):
        # sort by arrival
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0
        completed = 0
        n = len(processes)
        ready_queue = []
        visited = [False] * n

        while completed < n:
            # add arrived processes
            for i in range(n):
                if processes[i].arrival <= time and not visited[i] and
processes[i] not in ready_queue:
                    ready_queue.append(processes[i])

            # handle idle time
            if not ready_queue:
                next_arrival = min(p.arrival for i, p in
enumerate(processes) if not visited[i])
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                continue

            # pick highest priority
            selected = min(ready_queue, key=lambda p: (p.priority,
p.arrival))
            ready_queue.remove(selected)

            # mark as visited
            for i in range(n):
                if processes[i].pid == selected.pid:
                    visited[i] = True

            # run to completion
            start = time
            time += selected.burst

            # record results
            self.calculate_metrics(selected, time)
            self.gantt.append((selected.pid, start, time))

            completed += 1

        self.display("Priority Scheduling (Non-Preemptive)")
        self.plot_gantt("Priority (Non-Preemptive)")
```

Figure 12. Priority Scheduling Non-Preemptive Function

The Priority Scheduling (Non-Preemptive) algorithm selects the process with the highest priority in the ready queue and runs it until completion before moving to the next. A running process remains active until its completion because non-preemptive scheduling does not allow interruption from incoming higher priority processes. The system selects the process which arrived first when two processes have the same priority.

Moving on, the <priority_non_preemptive()> function initially resets Gantt charts and metrics, sorts the processes by arrival time, as well as initializes the CPU time. It has two main lists to manage the process execution, specifically the <ready_queue[]> holds processes that have arrived, and <visited[]> is a boolean list tracks which processes have already finished execution.

Then, a <while> loop is used to run the algorithm of Priority Non-Preemptive, this loop runs until all processes finish. Inside said loop is the main core of the function, which simulates the Priority Scheduling algorithm. For all processes whose arrival time is

less than or equal to the current CPU <time>, not yet in <visited[]>, and not in the <ready_queue[]>, they are appended into the <ready_queue[]>.

In the condition that the <ready_queue[]> is non-empty then the Priority Scheduling algorithm is simulated as follows. The <selected> variable stores the process with the lowest priority number retrieved using <min()> and lamda function with the criteria <(priority, arrival)> so that chosen processes are still dependent on arrival time. That process is then removed from the <ready_queue[]>.

Following that, a <for> loop marks it in <visited[]> by matching its <pid>, preventing it from being re-added and so preventing duplication. Thereafoter, the CPU clock's <start> is recorded, <time> is jumps depending on the process's full burst, plus its performance metrics is calculated using <calculate_metrics()> and the Gantt chart is populated.

For idle handling, a nested <if> condition checks if the <ready_queue[]> is empty, if so then the CPU clock <time> jumps to the nearest unvisited process arrival time, then immediately logs the idle period in the Gantt chart.

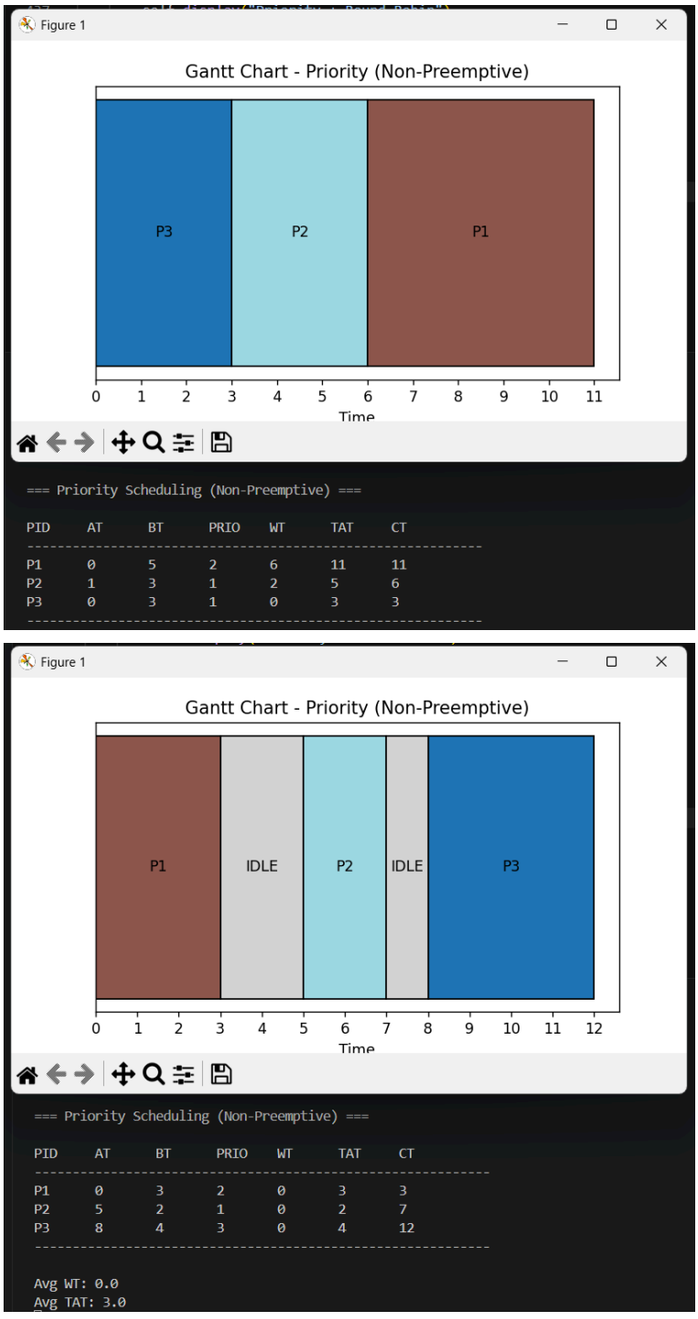


Figure 13. Priority Scheduling Non-Preemptive Test Cases

Test Case 1 and Test Case 2 visualizes the execution of the processes in terms of Non-Preemptive Priority Scheduling wherein the CPU scheduler sorts the processes

according to their arrival time and then prioritizes the processes tagged with the lower value. To elaborate, this CPU scheduler algorithm must still only choose the processes in the ready queue, next, it scans the whole processes to pinpoint the ones most prioritized. In the program, the most prioritized are the ones with the lower value, or rather from 0,1,2… and through this tag it can successfully take them from the ready queue then completely execute. Additionally, in Case 2 since certain processes arrive late, there is an additional idle time depicted on the Gantt chart.

Preemptive

```
    # priority preemptive
    def priority_preemptive(self):
        # setup remaining bursts
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0
        completed = 0
        n = len(processes)
        remaining_burst = [p.burst for p in processes]
        visited = [False] * n
        prev = None

        while completed < n:
            # find ready processes
            ready_indices = []
            for i in range(n):
                if processes[i].arrival <= time and not visited[i]:
                    ready_indices.append(i)

            # handle idle time
            if not ready_indices:
                next_arrival = None
                for i in range(n):
                    if not visited[i]:
                        if next_arrival is None or processes[i].arrival
< next_arrival:
                            next_arrival = processes[i].arrival
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                prev = None
                continue

            # pick highest priority
            best_index = ready_indices[0]
            for i in ready_indices:
                if (processes[i].priority, processes[i].arrival) <
(processes[best_index].priority, processes[best_index].arrival):
                    best_index = i

            # execute one unit
            p = processes[best_index]

            if prev != best_index:
                self.gantt.append([p.pid, time, time + 1])
            else:
                self.gantt[-1][2] += 1

            remaining_burst[best_index] -= 1
            time += 1
            prev = best_index

            # check completion
            if remaining_burst[best_index] == 0:
                visited[best_index] = True
                completed += 1
                self.calculate_metrics(p, time)

        self.display("Priority Scheduling (Preemptive)")
        self.plot_gantt("Priority (Preemptive)")
```

Figure 14. Priority Scheduling Preemptive Function

The Priority Scheduling (Preemptive) algorithm selects the process with the highest priority in the ready queue, however unlike the non-preemptive, a running process can be interrupted if ever a higher-priority process arrives. In the case that processes's priority are the same, the process that arrived first is chosen.

The <priority_preemptive()> function initially resets Gantt charts and metrics, sorts the processes by arrival time, as well as initializes the CPU time. Similar to SRT, it uses <remaining_burst[]> is a list that stores processes their remaining burst time since this function also runs tick by tick. The <visited[]> is a boolean list tracking processes with finished execution, while <prev> tracks the index of the process that ran in the previous tick later to be used to detect when a context switch happens later on.

Then, a <while> loop is used to run the algorithm of Priority Preemptive, this loop runs until all processes finish. Inside said loop, <ready_indices[]> is populated by indices of processes that have arrived or before the current CPU <time> and have not yet been marked as <visited>.

In the condition that <ready_indices[]> is empty, it means no process has arrived yet, and thus has a CPU idle gap. The program then iterates through all unvisited processes to find the one with the lowest arrival time, storing it in <next_arrival>, wherein then the Gantt chart is populated with "IDLE" from the current <time> to <next_arrival>. Also, the CPU clock <time> jumps forward, and <prev> is reset to <None>.

If the <ready_indices> is non-empty then the Priority Preemptive scheduling algorithm is simulated as follows. The first index in <ready_indices> is stored in <best_index>, then iterates through the rest of the procedure. If a process at index <i> has a lower <(priority, arrival)> pair than the current <best_index>. It subsequently replaces it, this is so that ties are still broken by arrival time. The process at <best_index> is then assigned to <p> and selected for execution.

Following that, since this algorithm runs tick by tick, the Gantt chart is handled the same way as SRT. The function checks if <prev> is equal to <best_index>. If they are the same, meaning the same process is still running, the last Gantt entry's end time is extended by one tick through <self.gantt[-1][2] += 1>. If they differ, meaning a context switch has occurred, a new Gantt entry is appended for the newly running process. Thereafter, <remaining_burst[best_index]> is decremented by one, <time> advances by one tick, and <prev> is updated to <best_index>.

Finally, after each tick, the scheduler checks if <remaining_burst[best_index]> has reached zero. If True then it is marked as finished by setting <visited[best_index]> to True, <completed> is incremented by one, and <calculate_metrics()> is called to compute its performance metrics. The <while> loop continues until <completed> equals <n>, meaning all processes have been fully executed.

Figure 15. Priority Scheduling Preemptive Test Cases

Test Case 1 and Test Case 2 visualizes the execution of the processes in terms of Preemptive Priority Scheduling wherein the CPU scheduler sorts the processes according to their arrival time and then prioritizes the processes tagged with the lower value. However, unlike the non-preemptive version, this preemptive algorithm means that a higher priority process can disrupt a currently running lower priority process. Additionally, in Case 2 since certain processes arrive late, there is an additional idle time depicted on the Gantt chart.

C.6. Priority Scheduling + Round Robin

```
    # priority with round robin
    def priority_round_robin(self, quantum):
        # initialize tracking
        self.reset()
        processes = sorted(self.processes, key=lambda p: (p.priority,
p.arrival))
        time = 0
        completed = 0
        n = len(processes)
        remaining_burst = [p.burst for p in processes]
        visited = [False] * n
        added = [False] * n
        ready_queue = []
```

```
        while completed < n:
            # add newly arrived
            for i in range(n):
                if processes[i].arrival <= time and not added[i]:
                    ready_queue.append(i)
                    added[i] = True

            # handle idle time
            if not ready_queue:
                next_arrival = min(
                    processes[i].arrival for i in range(n) if not
added[i]
                )
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                continue

            # pick highest priority group
            best_priority = min(processes[i].priority for i in
ready_queue)
            idx = next(i for i in ready_queue if processes[i].priority
== best_priority)
            ready_queue.remove(idx)

            # run time slice
            p = processes[idx]
            run_time = min(remaining_burst[idx], quantum)

            start = time
            time += run_time
            remaining_burst[idx] -= run_time

            # record execution
            self.gantt.append((p.pid, start, time))

            # add newly arrived after slice
            for i in range(n):
                if processes[i].arrival <= time and not added[i]:
                    ready_queue.append(i)
                    added[i] = True

            # check completion
            if remaining_burst[idx] == 0:
                visited[idx] = True
                completed += 1
                self.calculate_metrics(p, time)
            else:
                ready_queue.append(idx)

        self.display("Priority + Round Robin")
        self.plot_gantt("Priority + Round Robin")
```

Figure 16. Priority Scheduling with Round Robin Function

The Priority + Round Robin scheduling algorithm is a combination of both Priority Scheduling and Round Robin in which processes are grouped by their priority and then RR is applied by executing prioritized processes only by a given quantum time. In other words, higher-priority processes are always executed first (Silberschatz et al., 2013). But in the case that among processes of equal priority, CPU time is shared fairly through the quantum time slice (Verma, 2025).

To simulate the scheduling algorithm, the priority_round_robin() function initially resets the Gantt chart and metrics, sorts the processes by priority then by arrival time, as well as initializes the CPU time. It has three main lists to manage process execution, specifically the <remaining_burst[]> tracks how much burst time each process has left, <visited[]> is a boolean list tracking which processes have already finished execution, and lastly, <added[]> prevents duplicates of processes that have already been inserted in the <ready_queue[]>. The <ready_queue[]> stores indices instead of processes themselves, maintaining priority and arrival order throughout execution tick by tick as this function handles preemptiveness.

Then, a <while> loop is used to run the algorithm of Priority + RR and runs until all processes finish. Inside said loop, a <for> loop appends any process whose arrival time is less than or equal to the current CPU <time> into the <ready_queue[]>, then immediately marks it in <added[]> if not yet included.

In the condition that the <ready_queue[]> is empty, the CPU is considered idle. The function then finds the nearest next arrival time among processes not yet in <added[]>, logs the idle period in the Gantt chart, and jumps the CPU clock <time> forward using <continue>.

If the <ready_queue[]> is non-empty then the Priority + RR scheduling algorithm is simulated as follows. Firstly, the <best_priority> variable identifies the lowest priority number among all processes currently in the <ready_queue[]>, next, the first index in queue is retrieved by <next()> to ensure FIFO order, and then removed from the <ready_queue[]>.

Following that, the function checks the chosen process's <remaining_burst[]>. If the burst time is less than the quantum, then <run_time> stores the remaining burst time. Else, it stores the quantum time slice. The CPU clock's <start> is recorded, <time> is advanced, <remaining_burst[]> is decremented, and populates the Gantt chart.

Since the CPU <time> is updated, before re-running the <while> newly arrived processes are appended into the <ready_queue[]>. Furthermore, the function checks if the previous run process's <remaining_burst[]> is equal to 0. If True then it is included in <visited[]> and its performance metrics are calculated using <calculate_metrics()>. However, if the process still has remaining burst time, it is once again appended to the end of the <ready_queue[]>, contending again in the next loop based on its priority.

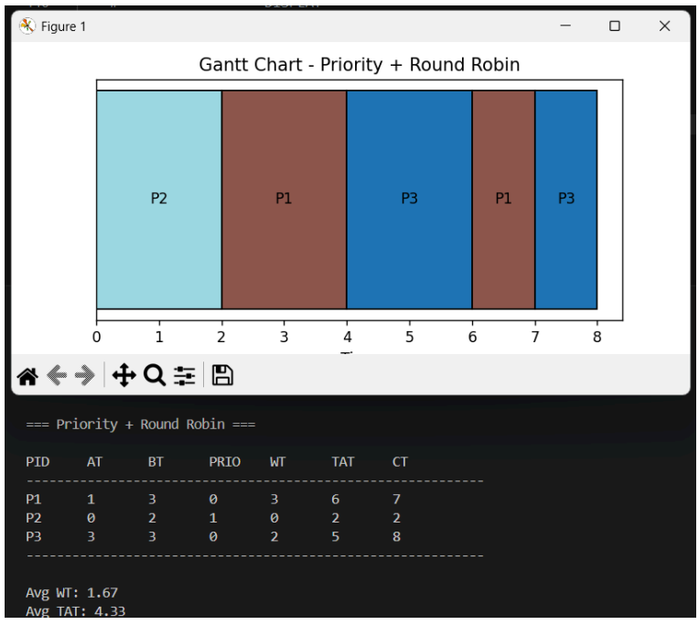


Figure 17. Priority Scheduling with Round Robin Test Cases

Test Case 1 and Test Case 2 visualizes the execution of the processes in terms of Preemptive Priority Scheduling wherein the CPU scheduler sorts the processes according to their arrival time and then prioritizes the processes tagged with the lower value. However, unlike the non-preemptive version, this preemptive algorithm means that a higher priority process can disrupt a currently running lower priority process. Still, unlike the previous version, this disruption only allots a pre-defined CPU quantum time. Hence, the CPU time is swapped between evenly prioritized process. Additionally, in Case 2 since certain processes arrive late, there is an additional idle time depicted on the Gantt chart.

D. Table Display and Gantt Chart

```
    # display results table
    def display(self, algo_name):
        print(f"\n=== {algo_name} ===")
        is_priority_algo = "Priority" in algo_name

        if is_priority_algo:
            print("\nPID\tAT\tBT\tPRIO\tWT\tTAT\tCT")
            print("-" * 60)
        else:
            print("\nPID\tAT\tBT\tWT\tTAT\tCT")
            print("-" * 50)

        total_wt = 0
        total_tat = 0

        for p in self.processes:
            if is_priority_algo:

print(f"{p.pid}\t{p.arrival}\t{p.burst}\t{p.priority}\t{p.waiting}\t{p.t
urnaround}\t{p.completion}")
            else:

print(f"{p.pid}\t{p.arrival}\t{p.burst}\t{p.waiting}\t{p.turnaround}\t{p
.completion}")
            total_wt += p.waiting
            total_tat += p.turnaround

        n = len(self.processes)
        print("-" * (60 if is_priority_algo else 50))
        print(f"\nAvg WT: {round(total_wt / n, 2)}")
        print(f"Avg TAT: {round(total_tat / n, 2)}")

    # plot gantt chart
    def plot_gantt(self, title):
```

```
        fig, ax = plt.subplots()
        import matplotlib.cm as cm
        import numpy as np

        unique_pids = list(set([pid for pid, _, _ in self.gantt if pid
!= "IDLE"]))
        color_map = {}
        colors = cm.tab20(np.linspace(0, 1, len(unique_pids)))
        for i, pid in enumerate(unique_pids):
            color_map[pid] = colors[i]

        idle_color = 'lightgray'

        for pid, start, end in self.gantt:
            label = str(pid)

            if pid == "IDLE":
                color = idle_color
            else:
                color = color_map[pid]

            ax.barh(0, end - start, left=start, color=color,
edgecolor='black')
            ax.text((start + end) / 2, 0, label, ha='center',
va='center')

        ax.set_xlabel("Time")
        ax.set_yticks([])
        ax.set_title(f"Gantt Chart - {title}")

        max_time = int(self.gantt[-1][2])
        ax.set_xticks(list(range(max_time + 1)))

        plt.show()
```

Figure 18. Gantt Chart via Matplotlib and Summary Table of Results

The <display()> and <plot_gantt()> are two outputs of <class Scheduler>, used to present the results of the user chosen scheduling algorithm. Both are called at the end of every scheduling algorithm function.

Firstly, the <display()> function accepts parameter <algo_name> for the name of the scheduling algorithm whose results are to be presented. The name is used for the header and the column has labels of PID, AT, BT, WT, TAT, and CT as process ID, burst, waiting, turnaround, and completion time and their respective rows are populated with <for> loop of the process's corresponding attributes. Then the <total_wt> and <total_tat> will be used for computing the averages, thereafter rounded to two decimal places using <round()>.

Secondly, the <plot_gantt()> function accepts parameter <title> to identify the scheduling algorithm name and is assigned as the Gantt chart's title. The function uses <matplotlib> to render a horizontal bar chart representing the CPU execution duration. A <for> loop iterates through <self.gantt>.

For each entry <pid>, a <start> time, and an <end> time, a horizontal bar is drawn using <ax.barh()>. Said bar spans from <start> to <end> using end - start as width. After that, <ax.text()> places the process ID label at the midpoint of each bar. Furthermore, the x-axis is labeled as "Time" and the y-axis ticks are hidden using <ax.set_yticks([])>.

To ensure that the x-axis only displays whole number intervals, <max_time> is derived from the last entry in <self.gantt>, and <ax.set_xticks()> is populated with a range from 0 to <max_time>. Finally, <plt.show()> renders the completed Gantt chart to the screen.

E. Comparative Analysis

Algorithm	Avg Waiting Time	Type	Analysis
FCFS	High	Non-Preemptive	Executes processes in the order they arrive
SJF	Low	Non-Preemptive	Selects the process with the smallest burst time among those that have arrived
SRT	Low(Optimal)	Preemptive	Selects the process with the smallest burst time among those that have arrived, and can disrupt current process
RR	Medium	Preemptive	Gives each process a fixed time quantum. Fair CPU time sharing
Priority (NP)	Varies	Non-Preemptive	Processes are selected based on the lowest priority number. Runs to completion
Priority (P)	Low	Low	Similar to SRT, but its basis is on burst time AND priority level
Priority + RR	Low	Preemptive	Sorts processes by priority and applies RR for those with the same priority level

Table 19. Comparative Analysis of CPU Scheduling Algorithm

The comparative results shown in the table reveals that pre-emptive CPU scheduling algorithms like SRT and Preemptive Priority have low waiting times. But, this responsiveness has higher context-switch overhead (Omar et al., 2021). Meanwhile, non-preemptive algorithms like FCFS are simple to implement but can suffer from the convoy effect as those long processes at the front of the ready queue can block smaller processes (Silberschatz et al., 2013). This causes inflation of both waiting and turnaround times. Additionally, the Priority + RR algorithm is found to be the most balanced as it can preserve priority ordering while providing fair CPU distribution among same level priority processes. This analysis comes from the result of Test Cases conducted in the case study. Still, choosing the most appropriate algorithm ultimately depends on the specific job characteristics as well as the system's tolerance for overhead.

III.Summary

The simulation of six CPU scheduling algorithms demonstrated how theoretical scheduling functions in actual system operation. CPU scheduling requires more than process sequence determination as it needs to balance three demands such as efficiency, fairness and responsiveness while using limited system resources it has. Every algorithm attempts to achieve maximum CPU utilization through different ways.

Throughout this case study, there were observed differences of each scheduling algorithm. It was found that scheduling depends on process attributes which include burst time and arrival time. Shortest Job First (SJF) and Shortest Remaining Time (SRT) scheduling algorithms reduce average waiting time as they give priority to shorter processes. The system achieves better efficiency through this algorithm. However, this favors shorter processes while longer processes suffer when shorter jobs arrive continuously. Meanwhile, First-Come, First-Served (FCFS) scheduling treats all processes equally as its basis is on their arrival time, but has poor responsiveness because long processes create longer delays for all subsequent processes.

In introspection, the use of preemptive algorithms like SRT and Preemptive Priority Scheduling reveals the need to focus on responsiveness. Because it is found that the algorithms that allow current operations to be interrupted achieve better turnaround time and waiting time. However, this advantage requires systems to handle more context switches which creates extra overhead and so decreases system performance.

Nowadays, modern operating systems depend on CPU scheduling because it allows multiple processes to share CPU time efficiently. Especially now that users operate multiple applications simultaneously. For example, Chrome web browser, Spotify background services and softwares like Microsoft Word, all running all at one. The complexity even further worsens as modern multi-core microprocessors run with high performance yet they need scheduling algorithms to choose which process will receive CPU resources during specific times, and must perform fast job switching between them. Ultimately, CPU scheduling serves as an essential operating system component because it determines how computer systems perform their tasks while treating users and maintaining system reliability in real-world applications.

Even now, at a theoretical level, there were some challenges faced in crating all six CPU scheduling algorithms. The first challenge faced in crafting the CPU scheduling algorithm simulator for this study is managing preemptive behavior and calculating scheduling metrics. Preemptive algorithms require a thorough monitoring of remaining burst times and hence must take note of context switches. With these, the complexity increased because the program needed to track changing states while maintaining correct waiting time, turnaround time and completion time calculations. The second challenge is the correct management of CPU idle periods throughout the CPU operation. It required multiple checks, wherein the simulator must monitor future arrival times while it adjusted the CPU clock to synchronize Gantt chart display. These challenges were resolved using the flexibility of array[], as the simulator can immediately track the state of each process.

To conclude, the case study of CPU scheduling reveals that no single CPU scheduling algorithm can achieve optimal performance in all situations. Every algorithm establishes its own balance depending on the specific requirements of the workload. Still, however, it was mainly discovered that contemporary operating systems maintain stable performance by using multiple methods which enable them to adjust their operations based on actual environmental conditions.

IV.References

GeeksforGeeks. (2026, April 24). *CPU scheduling in operating systems*. GeeksforGeeks. Retrieved April 25, 2026, from https://www.geeksforgeeks.org/operating-systems/cpu-scheduling-in-operating-systems/

Omar, H. K., Jihad, K. H., & Hussein, S. F. (2021). Comparative analysis of the essential CPU scheduling algorithms. *Bulletin of Electrical Engineering and Informatics*, *10*(5), 2742-2750.

Seemakuthi, S., Siriki, V. A., & Lydia, E. L. (2015). A Review on Various Scheduling Algorithms. *Int. J. Sci. Eng. Res*, *6*, 769-779.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating system concepts essentials*. Wiley Publishing.

Verma, A. (2025, October 1). *CPU scheduling in operating systems*. Intellipaat. https://intellipaat.com/blog/cpu-scheduling-in-operating-system/

Github Repository Link:
https://github.com/NJoy1023/OS-Case-Study

SOURCE CODE

```
import matplotlib.pyplot as plt


class Process:
    # holds individual process data
    def __init__(self, pid, arrival, burst, priority=0):
        self.pid = pid
        self.arrival = arrival
        self.burst = burst
        self.priority = priority
        self.waiting = 0
        self.turnaround = 0
        self.completion = 0


class Scheduler:
    # manages all scheduling algorithms
    def __init__(self, processes):
        self.processes = processes
        self.gantt = []

    # reset all process metrics
    def reset(self):
        self.gantt = []
        for p in self.processes:
            p.waiting = 0
            p.turnaround = 0
            p.completion = 0

    # compute waiting and turnaround
    def calculate_metrics(self, p, completion_time):
        p.completion = completion_time
        p.turnaround = p.completion - p.arrival
        p.waiting = p.turnaround - p.burst

    # first come first served
    def fcfs(self):
        # sort by arrival
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0

        for p in processes:
            # handle idle time
            if time < p.arrival:
                self.gantt.append(("IDLE", time, p.arrival))
                time = p.arrival

            # execute process
            start = time
            time += p.burst
            finish = time

            # store results
            self.calculate_metrics(p, finish)
            self.gantt.append((p.pid, start, finish))

        self.display("FCFS")
        self.plot_gantt("FCFS")

    # shortest job first nonpreemptive
    def sjf_non_preemptive(self):
        # initialize tracking
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0
        completed = 0
        n = len(processes)
        ready_queue = []
        visited = [False] * n
```

```
        while completed < n:
            # add arrived processes
            for i in range(n):
                if processes[i].arrival <= time and not visited[i]:
                    ready_queue.append(processes[i])
                    visited[i] = True

            # handle idle cpu
            if not ready_queue:
                next_arrival = None
                for i in range(n):
                    if not visited[i]:
                        if next_arrival is None or processes[i].arrival <
next_arrival:
                            next_arrival = processes[i].arrival
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                continue

            # pick shortest job
            shortest = min(ready_queue, key=lambda p: p.burst)
            ready_queue.remove(shortest)

            # run to completion
            start = time
            time += shortest.burst

            # record metrics
            self.calculate_metrics(shortest, time)
            self.gantt.append((shortest.pid, start, time))

            completed += 1

        self.display("SJF (Non-Preemptive)")
        self.plot_gantt("SJF")

    # shortest remaining time preemptive
    def srt(self):
        # setup remaining bursts
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0
        completed = 0
        n = len(processes)
        remaining_burst = []
        for p in processes:
            remaining_burst.append(p.burst)
        visited = [False] * n
        prev = None

        while completed < n:
            # find ready processes
            ready_indices = []
            for i in range(n):
                if processes[i].arrival <= time and not visited[i]:
                    ready_indices.append(i)

            # handle idle time
            if not ready_indices:
                next_arrival = None
                for i in range(n):
                    if not visited[i]:
                        if next_arrival is None or processes[i].arrival <
next_arrival:
                            next_arrival = processes[i].arrival
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                continue

            # pick shortest remaining
            best_index = ready_indices[0]
            for i in ready_indices:
                if remaining_burst[i] < remaining_burst[best_index]:
                    best_index = i
```

```
            # execute one unit
            p = processes[best_index]

            if prev != best_index:
                self.gantt.append([p.pid, time, time + 1])
            else:
                self.gantt[-1][2] += 1

            remaining_burst[best_index] -= 1
            time += 1
            prev = best_index

            # check for completion
            if remaining_burst[best_index] == 0:
                visited[best_index] = True
                completed += 1
                self.calculate_metrics(p, time)

        self.display("SRT (Preemptive SJF)")
        self.plot_gantt("SRT")

    # round robin with quantum
    def round_robin(self, quantum):
        # initialize queues
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0
        completed = 0
        n = len(processes)
        remaining_burst = [p.burst for p in processes]
        visited = [False] * n
        ready_queue = []
        added = [False] * n

        # add initial processes
        for i in range(n):
            if processes[i].arrival <= time:
                ready_queue.append(i)
                added[i] = True

        while completed < n:
            # handle empty queue
            if not ready_queue:
                next_arrival = None
                for i in range(n):
                    if not added[i]:
                        if next_arrival is None or processes[i].arrival <
next_arrival:
                            next_arrival = processes[i].arrival
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                for i in range(n):
                    if processes[i].arrival <= time and not added[i]:
                        ready_queue.append(i)
                        added[i] = True
                continue

            # get next process
            idx = ready_queue.pop(0)
            p = processes[idx]

            # determine run time
            if remaining_burst[idx] < quantum:
                run_time = remaining_burst[idx]
            else:
                run_time = quantum

            # execute time slice
            start = time
            time += run_time
            remaining_burst[idx] -= run_time

            # record execution
            self.gantt.append((p.pid, start, time))
```

```
            # add newly arrived
            for i in range(n):
                if processes[i].arrival <= time and not added[i]:
                    ready_queue.append(i)
                    added[i] = True

            # check completion
            if remaining_burst[idx] == 0:
                visited[idx] = True
                completed += 1
                self.calculate_metrics(p, time)
            else:
                ready_queue.append(idx)

        self.display("Round Robin")
        self.plot_gantt("Round Robin")

    # priority nonpreemptive
    def priority_non_preemptive(self):
        # sort by arrival
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0
        completed = 0
        n = len(processes)
        ready_queue = []
        visited = [False] * n

        while completed < n:
            # add arrived processes
            for i in range(n):
                if processes[i].arrival <= time and not visited[i] and
processes[i] not in ready_queue:
                    ready_queue.append(processes[i])

            # handle idle time
            if not ready_queue:
                next_arrival = min(p.arrival for i, p in enumerate(processes)
if not visited[i])
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                continue

            # pick highest priority
            selected = min(ready_queue, key=lambda p: (p.priority, p.arrival))
            ready_queue.remove(selected)

            # mark as visited
            for i in range(n):
                if processes[i].pid == selected.pid:
                    visited[i] = True

            # run to completion
            start = time
            time += selected.burst

            # record results
            self.calculate_metrics(selected, time)
            self.gantt.append((selected.pid, start, time))

            completed += 1

        self.display("Priority Scheduling (Non-Preemptive)")
        self.plot_gantt("Priority (Non-Preemptive)")

    # priority preemptive
    def priority_preemptive(self):
        # setup remaining bursts
        self.reset()
        processes = sorted(self.processes, key=lambda p: p.arrival)
        time = 0
        completed = 0
        n = len(processes)
        remaining_burst = [p.burst for p in processes]
        visited = [False] * n
```

```
        prev = None

        while completed < n:
            # find ready processes
            ready_indices = []
            for i in range(n):
                if processes[i].arrival <= time and not visited[i]:
                    ready_indices.append(i)

            # handle idle time
            if not ready_indices:
                next_arrival = None
                for i in range(n):
                    if not visited[i]:
                        if next_arrival is None or processes[i].arrival <
next_arrival:
                            next_arrival = processes[i].arrival
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                prev = None
                continue

            # pick highest priority
            best_index = ready_indices[0]
            for i in ready_indices:
                if (processes[i].priority, processes[i].arrival) <
(processes[best_index].priority, processes[best_index].arrival):
                    best_index = i

            # execute one unit
            p = processes[best_index]

            if prev != best_index:
                self.gantt.append([p.pid, time, time + 1])
            else:
                self.gantt[-1][2] += 1

            remaining_burst[best_index] -= 1
            time += 1
            prev = best_index

            # check completion
            if remaining_burst[best_index] == 0:
                visited[best_index] = True
                completed += 1
                self.calculate_metrics(p, time)

        self.display("Priority Scheduling (Preemptive)")
        self.plot_gantt("Priority (Preemptive)")

    # priority with round robin
    def priority_round_robin(self, quantum):
        # initialize tracking
        self.reset()
        processes = sorted(self.processes, key=lambda p: (p.priority,
p.arrival))
        time = 0
        completed = 0
        n = len(processes)
        remaining_burst = [p.burst for p in processes]
        visited = [False] * n
        added = [False] * n
        ready_queue = []

        while completed < n:
            # add newly arrived
            for i in range(n):
                if processes[i].arrival <= time and not added[i]:
                    ready_queue.append(i)
                    added[i] = True

            # handle idle time
            if not ready_queue:
                next_arrival = min(
                    processes[i].arrival for i in range(n) if not added[i]
```

```
                )
                self.gantt.append(("IDLE", time, next_arrival))
                time = next_arrival
                continue

            # pick highest priority group
            best_priority = min(processes[i].priority for i in ready_queue)
            idx = next(i for i in ready_queue if processes[i].priority ==
best_priority)
            ready_queue.remove(idx)

            # run time slice
            p = processes[idx]
            run_time = min(remaining_burst[idx], quantum)

            start = time
            time += run_time
            remaining_burst[idx] -= run_time

            # record execution
            self.gantt.append((p.pid, start, time))

            # add newly arrived after slice
            for i in range(n):
                if processes[i].arrival <= time and not added[i]:
                    ready_queue.append(i)
                    added[i] = True

            # check completion
            if remaining_burst[idx] == 0:
                visited[idx] = True
                completed += 1
                self.calculate_metrics(p, time)
            else:
                ready_queue.append(idx)

        self.display("Priority + Round Robin")
        self.plot_gantt("Priority + Round Robin")

    # display results table
    def display(self, algo_name):
        print(f"\n=== {algo_name} ===")
        is_priority_algo = "Priority" in algo_name

        if is_priority_algo:
            print("\nPID\tAT\tBT\tPRIO\tWT\tTAT\tCT")
            print("-" * 60)
        else:
            print("\nPID\tAT\tBT\tWT\tTAT\tCT")
            print("-" * 50)

        total_wt = 0
        total_tat = 0

        for p in self.processes:
            if is_priority_algo:

print(f"{p.pid}\t{p.arrival}\t{p.burst}\t{p.priority}\t{p.waiting}\t{p.turnaro
und}\t{p.completion}")
            else:

print(f"{p.pid}\t{p.arrival}\t{p.burst}\t{p.waiting}\t{p.turnaround}\t{p.compl
etion}")
            total_wt += p.waiting
            total_tat += p.turnaround

        n = len(self.processes)
        print("-" * (60 if is_priority_algo else 50))
        print(f"\nAvg WT: {round(total_wt / n, 2)}")
        print(f"Avg TAT: {round(total_tat / n, 2)}")

    # plot gantt chart
    def plot_gantt(self, title):
        fig, ax = plt.subplots()
        import matplotlib.cm as cm
```

```
        import numpy as np

        unique_pids = list(set([pid for pid, _, _ in self.gantt if pid !=
"IDLE"]))
        color_map = {}
        colors = cm.tab20(np.linspace(0, 1, len(unique_pids)))
        for i, pid in enumerate(unique_pids):
            color_map[pid] = colors[i]

        idle_color = 'lightgray'

        for pid, start, end in self.gantt:
            label = str(pid)

            if pid == "IDLE":
                color = idle_color
            else:
                color = color_map[pid]

            ax.barh(0, end - start, left=start, color=color,
edgecolor='black')
            ax.text((start + end) / 2, 0, label, ha='center', va='center')

        ax.set_xlabel("Time")
        ax.set_yticks([])
        ax.set_title(f"Gantt Chart - {title}")

        max_time = int(self.gantt[-1][2])
        ax.set_xticks(list(range(max_time + 1)))

        plt.show()


# get user input for processes
def get_input(choice):
    needs_priority = choice in [5, 6, 7]

    while True:
        try:
            n = int(input("Enter number of processes (>=3): "))
            if n >= 3:
                break
            print("Minimum is 3 processes.")
        except ValueError:
            print("Please enter a valid integer.")

    processes = []

    for i in range(n):
        print(f"\nProcess {i + 1}")
        pid = input("Process Identifier: ")

        while True:
            try:
                bt = int(input("Burst Time: "))
                at = int(input("Arrival Time: "))

                if needs_priority:
                    pr = int(input("Priority (Lower Value = Higher Priority):
"))
                else:
                    pr = 0

                break
            except ValueError:
                print("Please enter valid integers.")

        processes.append(Process(pid, at, bt, pr))

    return processes


# main program entry point
def main():
    print("\nChoose Scheduling Algorithm:")
```

```
    print("1. FCFS")
    print("2. SJF (Non-Preemptive)")
    print("3. SRT (Preemptive SJF)")
    print("4. Round Robin")
    print("5. Priority Scheduling (Non-Preemptive)")
    print("6. Priority Scheduling (Preemptive)")
    print("7. Priority + Round Robin")

    while True:
        try:
            choice = int(input("Enter choice: "))
            if choice in range(1, 8):
                break
            print("Please enter a number between 1 and 7.")
        except ValueError:
            print("Please enter a valid integer.")

    processes = get_input(choice)
    scheduler = Scheduler(processes)

    if choice == 1:
        scheduler.fcfs()
    elif choice == 2:
        scheduler.sjf_non_preemptive()
    elif choice == 3:
        scheduler.srt()
    elif choice == 4:
        while True:
            try:
                quantum = int(input("\nEnter Time Quantum: "))
                break
            except ValueError:
                print("Please enter a valid integer.")
        scheduler.round_robin(quantum)
    elif choice == 5:
        scheduler.priority_non_preemptive()
    elif choice == 6:
        scheduler.priority_preemptive()
    elif choice == 7:
        while True:
            try:
                quantum = int(input("Enter Time Quantum: "))
                break
            except ValueError:
                print("Please enter a valid integer.")
        scheduler.priority_round_robin(quantum)
    else:
        print("Invalid choice")


# run simulation loop
while True:
    main()
    print("\nRun another simulation?")
    print("1. Yes")
    print("2. No")

    while True:
        try:
            again = int(input("Enter choice: "))
            if again in [1, 2]:
                break
            print("Please enter 1 or 2 only.")
        except ValueError:
            print("Please enter a valid integer.")

    if again == 2:
        print("Exiting program...")
        break
```

Github Repository Link:
https://github.com/NJoy1023/OS-Case-Study